

OpenClaw Natural Interaction + Multimodal Visual Understanding + SLAM Navigation + Visual Following

OpenClaw Natural Interaction + Multimodal Visual Understanding + SLAM Navigation + Visual Following

1. Course Content
2. Preparation
3. Case Demonstration
 - 3.1 Record a Semantic Position
5. Source Code Analysis
 - 5.1 TargetTrack
 - 5.2 visual_track.py
6. Parameter Debugging

1. Course Content

- Use OpenClaw to complete a compound long-flow task: navigate to the target area according to the instruction semantics, then track the target

2. Preparation

 Important

- Complete the prerequisite course content of [OpenClaw Natural Interaction + SLAM Navigation]

```
ros2 launch microros_v2_bringup openclaw_service.launch.py use_track:=true use_nav:=true
```

[illegible]

- Use the 2D Pose Estimate tool to initialize the robot's position in the map



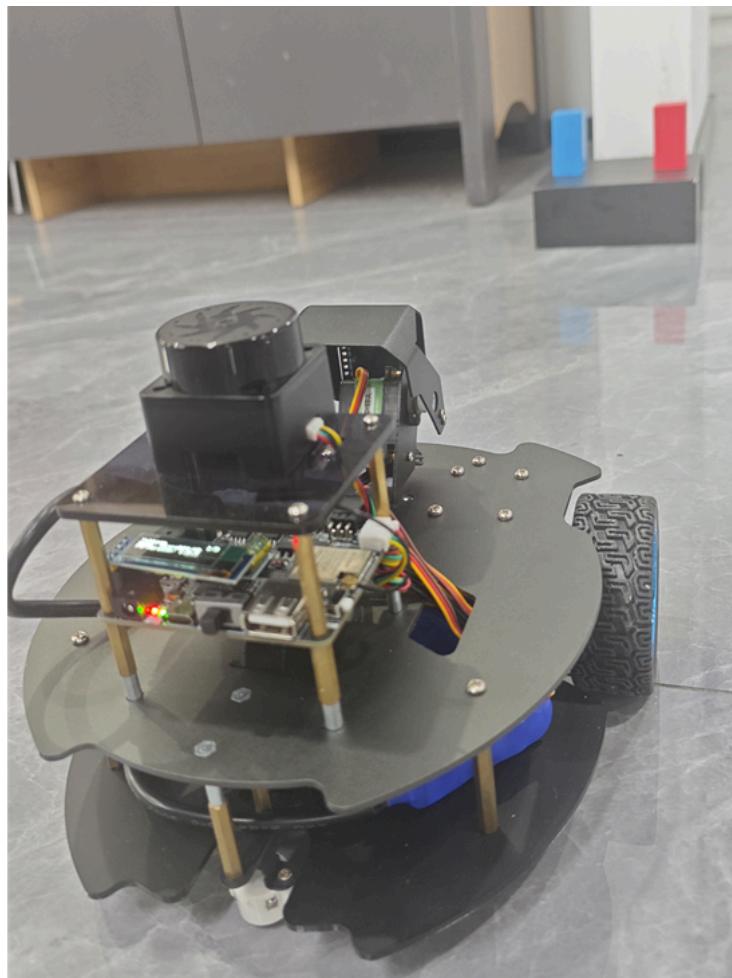
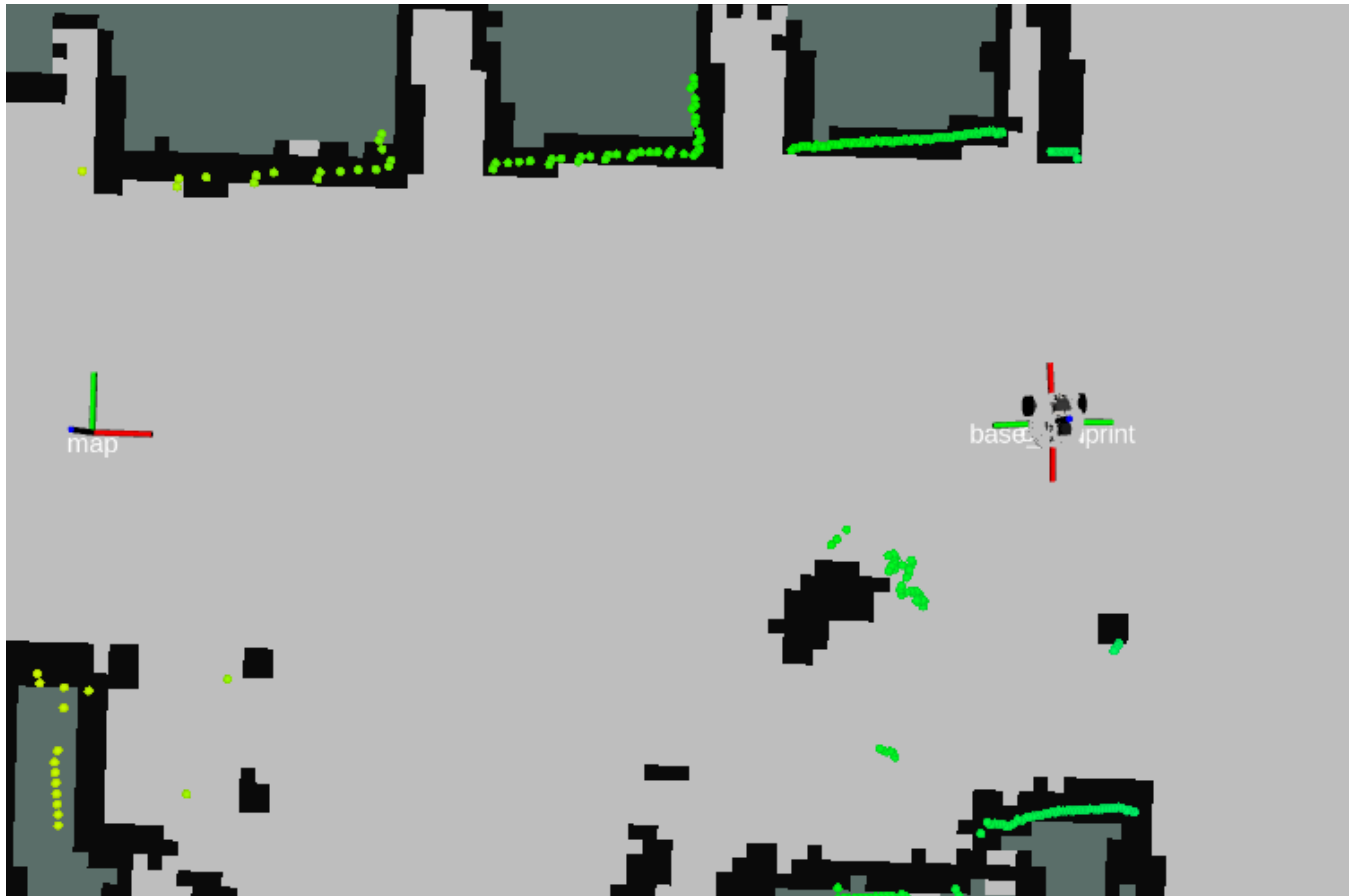
3. Case Demonstration

💡 Tip

- You can write the commands and scenarios according to your own needs. The following is a case demonstration. You can choose any interaction method; here we demonstrate with Web Chat on the web page as an example

3.1 Record a Semantic Position

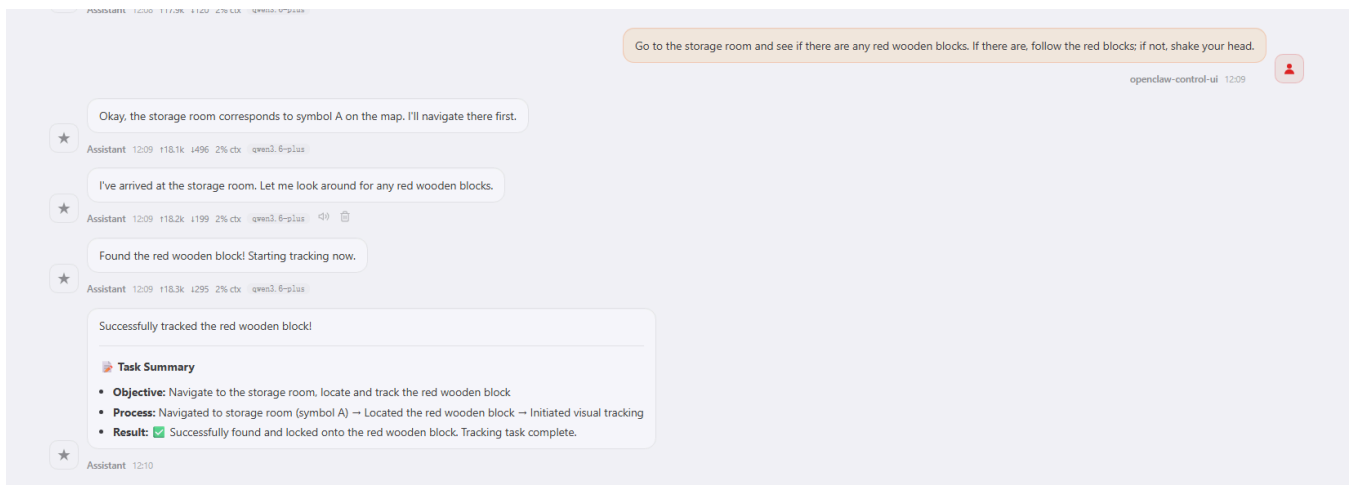
- First record a position on the map and set up a simple scene with colored materials



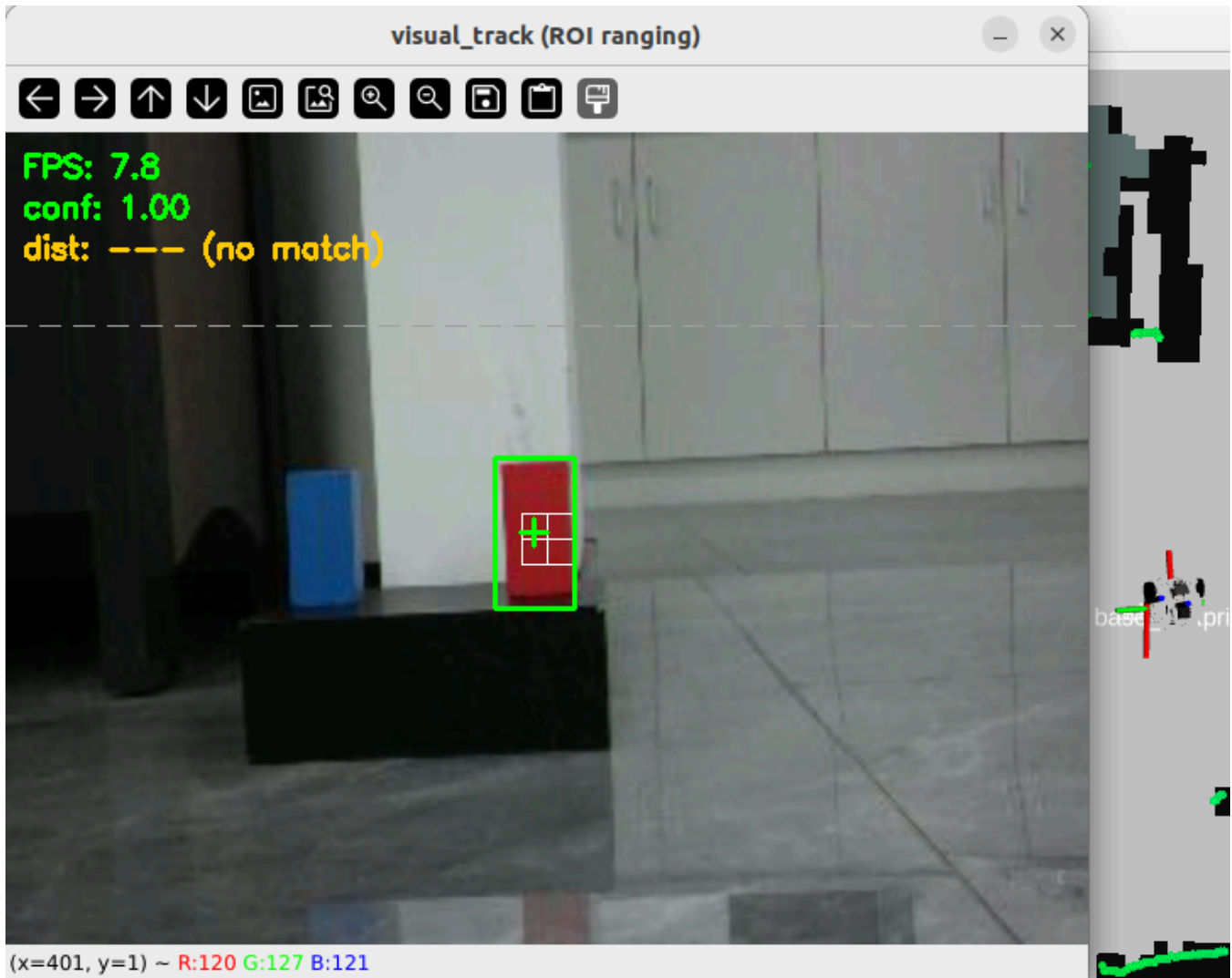
```
robot_control call-tool RecordMapLocation --name 货物间 --symbol A
```

```
yahboom@yahboom:~$ robot_control call-tool RecordMapLocation --name 货物间 --symbol A
{
  "execution_result": "success",
  "reason": ""
}
```

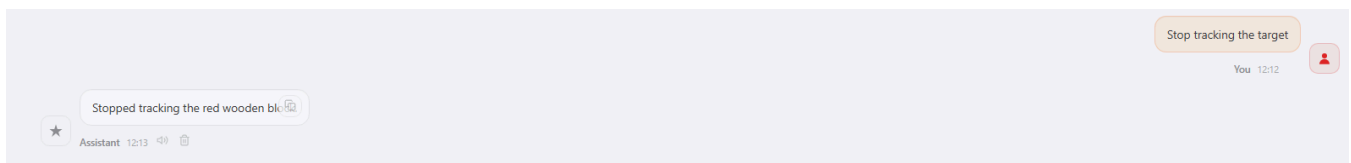
- Reference scenario example:
 - Go to the cargo room, check whether there is a red wooden block. If there is, track the red wooden block; if not, shake the head to indicate



- When the red wooden block is moved, the robot will follow and lock onto the target through the pan-tilt camera and chassis. The robot uses the initially observed object Bbox area as a reference. When the object becomes smaller in the frame, it means the object is moving away, and the robot will control the front-back distance to follow. When the object moves left or right, the robot will use both the pan-tilt and the chassis to track the target



- When you need to cancel tracking the target, you can issue a command to cancel the target tracking



5. Source Code Analysis

- Source Code Path

```
$HOME/microros_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

5.1 TargetTrack

`TargetTrack()` is the visual tracking entry function in `action_controller.py`. It is responsible for starting or canceling the target tracking mode. The function distinguishes two operations by the type of the `x1_or_cmd` parameter: when the string "cancel" is passed, tracking is canceled; when an integer (the x1 coordinate of the top-left corner of the bbox) is passed, tracking is started. The overall flow is divided into two branches:

Cancel tracking branch (`x1_or_cmd` is a string):

1. **Command validation:** Only accepts the "cancel" command; other strings return an error message
2. **Disable follow:** First calls `SetTracking(enable=False)` to disable the pan-tilt/chassis tracking follow, to prevent the chassis from still moving during cancellation
3. **Cancel tracking and reset:** Calls the `CancelTrack()` service to notify the visual tracking node to stop tracking and reset the pan-tilt pose

Start tracking branch (`x1_or_cmd` is an integer):

1. **Parameter completeness check:** `y1`, `x2`, `y2` must all be provided, otherwise a missing-parameter error is returned
2. **Initialize the tracking target:** Calls the `startTrack` service to pass the target bbox coordinates (`x1`, `y1`, `x2`, `y2`) to the visual tracking node for target feature extraction
3. **Enable pan-tilt/chassis follow:** Calls `SetTracking(enable=True)` to enable pan-tilt PID tracking and chassis follow, so that the pan-tilt continuously aims at the target and the chassis follows to turn

```
def TargetTrack(self, x1_or_cmd, y1:int=None, x2:int=None, y2:int=None):

    # Cancel tracking mode
    if isinstance(x1_or_cmd, str):
        if x1_or_cmd != "cancel":
            self.get_logger().warn(Fore.YELLOW+f"Invalid command for TargetTrack: Use 'cancel' to stop tracking{x1_or_cmd}"+Fore.RESET)
            return [False, "Invalid command for TargetTrack. Use 'cancel' to stop tracking."]

    if self.set_tracking_client.wait_for_service(timeout_sec=5.0):
        self.set_tracking_client.call(SetTracking.Request(enable=False))
    else:
        self.get_logger().warn("SetTracking service not available when cancelling")

    if not self.cancel_track_client.wait_for_service(timeout_sec=5.0):
        self.get_logger().error("CancelTrack service not available")
        return [False, "CancelTrack service not available"]
    self.cancel_track_client.call(CancelTrack.Request())
    return [True, "Tracking cancelled and arm reset"]

    # Start tracking mode
    x1: int = x1_or_cmd
    if None in (y1, x2, y2):
        self.get_logger().error("TargetTrack: x1, y1, x2, y2 are all required to start tracking")
        return [False, "Missing bounding box parameters"]

    # Start tracking mode
    if not self.start_track_client.wait_for_service(timeout_sec=5.0):
        self.get_logger().error("StartTrack service not available")
        return [False, "Tracking service not available"]
    result: StartTrack.Response = self.start_track_client.call(StartTrack.Request(x1=x1, y1=y1, x2=x2, y2=y2))
    if not result.success:
        self.get_logger().error("Failed to initialize tracking target")
```

```

        return [False, "Tracking failed"]

    # Request gimbal and chassis tracking services /visual_track/set_tracking
    if not self.set_tracking_client.wait_for_service(timeout_sec=5.0):
        self.get_logger().error("SetTracking service not available")
        if self.cancel_track_client.wait_for_service(timeout_sec=5.0):
            self.cancel_track_client.call(CancelTrack.Request())
            return [False, "SetTracking service not available"]
    set_result: SetTracking.Response =
self.set_tracking_client.call(SetTracking.Request(enable=True))
    if not set_result.success:
        self.get_logger().error("Failed to enable gimbal/chassis tracking")
        if self.cancel_track_client.wait_for_service(timeout_sec=5.0):
            self.cancel_track_client.call(CancelTrack.Request())
            return [False, "Failed to enable gimbal/chassis tracking"]

    return [True, "Tracking started"]

```

5.2 visual_track.py

Source Code Path

```
$HOME/microros_ws/src/microros_v2_bringup/microros_v2_bringup/visual_track.py
```

`visual_track.py` is the core implementation node of visual tracking, adopting a **dual-loop control architecture of fast and slow loops**: the fast loop keeps the target centered through the pan-tilt servo, and the slow loop achieves body alignment and distance keeping through chassis movement.

1. `_update_bbox_distance`: Estimates the relative distance by the bbox circumscribed circle area according to "near large, far small". The median of the area over the first N frames is used as the reference. For subsequent frames, `ratio = sqrt(reference area / current area)` is calculated; >1 means farther than the reference and needs to move forward, <1 means closer than the reference and needs to move backward. After EMA smoothing, it is used for distance keeping.
2. `track(cx, cy)`: The main control loop entry. Calculates the target pixel error (`error_x, error_y`) and splits into two modes:
 - **ptz mode**: The fast loop maps the normalized pixel error to a pan-tilt step angle through PID, updates the absolute angle, and publishes it to the servo; it also compensates for the feedforward offset caused by body rotation. The slow loop calls `update_chassis` for chassis follow and distance keeping.
 - **no_ptz mode**: No pan-tilt; directly calls `update_chassis_no_ptz`, and the follow is completed by chassis rotation + forward/backward movement.
3. `update_chassis`: Chassis control in ptz mode — `_compute_body_follow` drives the chassis to turn toward the target based on the pan-tilt angle deviation, and `_compute_distance_keeping_bbox` controls forward/backward movement based on the bbox distance ratio. It is only executed when the front of the vehicle is basically aligned.
4. `update_chassis_no_ptz`: Chassis control in no_ptz mode — `_compute_body_follow_pixel` drives the

chassis to rotate based on the horizontal pixel error, and `_compute_distance_keeping_bbox` controls forward/backward movement.

5. `_compute_body_follow` / `_compute_body_follow_pixel`: Both use PD control with hysteresis start/stop logic (start when the error exceeds the threshold, stop when it falls below the threshold) to avoid chassis jitter from frequent start/stop.

```
def _update_bbox_distance(self, x1, y1, x2, y2):
    """Estimate the relative distance by the bbox circumscribed circle area using near-
    large-far-small (works for both ptz / no_ptz).

    The median area over the first bbox_ref_frames frames after capturing the target is
    used as the
    reference; the area is inversely proportional to the square of the distance,
    relative distance = sqrt(reference area / current area), >1 means farther, <1 means
    closer.
    """
    bw = max(1.0, float(x2) - float(x1))
    bh = max(1.0, float(y2) - float(y1))
    area = math.pi * (bw * bw + bh * bh) / 4.0 # Circumscribed circle area = pi*
    (diagonal/2)^2

    if self.bbox_ref_area is None:
        self.bbox_ref_samples.append(area)
        if len(self.bbox_ref_samples) >= self.bbox_ref_frames:
            self.bbox_ref_area = float(np.median(self.bbox_ref_samples))
            self.bbox_area_filt = self.bbox_ref_area
            self.bbox_dist_ratio = 1.0
        return

    # EMA smooth the current area to suppress detection box jitter
    a = self.bbox_area_smooth
    self.bbox_area_filt = a * area + (1.0 - a) * self.bbox_area_filt
    ratio = math.sqrt(self.bbox_ref_area / max(self.bbox_area_filt, 1.0))
    self.bbox_dist_ratio = self.clamp(ratio, 1.0 / self.bbox_ratio_max,
    self.bbox_ratio_max)

def track(self, cx, cy):
    now = time.monotonic()
    if self.last_control_time is None:
        dt = 1.0 / 30.0
    else:
        dt = self.clamp(now - self.last_control_time, 1e-3, 0.2)
    self.last_control_time = now

    half_w = max(1.0, self.image_width / 2.0)
    half_h = max(1.0, self.image_height / 2.0)

    error_x = cx - self.target_x # >0: target is to the right
    error_y = cy - self.target_y # >0: target is below

    # ---- no_ptz mode: no pan-tilt, follow the target only by chassis rotation +
    forward/backward movement ----
```



```

if not self.use_ptz:
    self.update_chassis_no_ptz(error_x, half_w, dt)
    return

# Body yaw change  $\Delta\text{yaw} = \text{last\_angular\_z} * \text{dt}$ , the servo needs to compensate
synchronously  $\Delta\text{current\_x} = \text{sign\_x} * \Delta\text{yaw}$ 
if abs(self.last_angular_z) > 1e-6:
    body_yaw_deg = math.degrees(self.last_angular_z * dt)
    self.current_x = self.clamp(
        self.current_x + self.body_ff_gain * self.sign_x * body_yaw_deg,
        self.servo1_min, self.servo1_max)

aligned_x = abs(error_x) <= self.pixel_tolerance
aligned_y = abs(error_y) <= self.pixel_tolerance

# ---- Fast loop (camera): keep the target centered by pixel feedback, reset the
corresponding PID for the aligned axis ----
if aligned_x:
    self.pid_x.reset()
    step_x = 0.0
else:
    # Normalized error -> PID -> clamped step angle
    norm_x = self.clamp(error_x / half_w, -1.0, 1.0)
    out_x = self.clamp(self.pid_x.compute(norm_x, 0.0), -1.0, 1.0)
    step_x = out_x * self.max_step_xy

if aligned_y:
    self.pid_y.reset()
    step_y = 0.0
else:
    norm_y = self.clamp(error_y / half_h, -1.0, 1.0)
    out_y = self.clamp(self.pid_y.compute(norm_y, 0.0), -1.0, 1.0)
    step_y = out_y * self.max_step_xy

# Update the absolute angle and clamp it within the servo travel range
self.current_x = self.clamp(self.current_x + self.sign_x * step_x, self.servo1_min,
self.servo1_max)
self.current_y = self.clamp(self.current_y + self.sign_y * step_y, self.servo2_min,
self.servo2_max)

self.publish_ptz(self.current_x, self.current_y)
# ---- Slow loop (chassis): body follow (facing the target) + distance keeping (x
linear velocity) ----
self.update_chassis(dt)

def update_chassis(self, dt):
    if self.enable_body_follow:
        angular_z = self._compute_body_follow(dt)
    else:
        self.body_follow_armed = False
        self.prev_servo_err = 0.0
        angular_z = 0.0

```

```

# ---- Linear velocity: distance keeping (only move forward/backward when the front
is basically aligned) ----
if self.enable_distance_keeping:
    aligned = abs(self.current_x) <= self.distance_align_deg
    linear_x = self._compute_distance_keeping_bbox(dt, aligned)
else:
    self.prev_dist_err = 0.0
    linear_x = 0.0

self.publish_cmd_vel(linear_x, angular_z)
self.last_angular_z = angular_z
self.last_linear_x = linear_x

def update_chassis_no_ptz(self, error_x, half_w, dt):
    """Chassis control in no_ptz mode: angular velocity is driven by the horizontal
    pixel error, linear velocity does distance keeping."""
    if self.enable_body_follow:
        angular_z = self._compute_body_follow_pixel(error_x, half_w, dt)
    else:
        self.body_follow_armed = False
        self.prev_servo_err = 0.0
        angular_z = 0.0

    if self.enable_distance_keeping:
        aligned = abs(error_x) <= self.no_ptz_align_px
        linear_x = self._compute_distance_keeping_bbox(dt, aligned)
    else:
        self.prev_dist_err = 0.0
        linear_x = 0.0

    self.publish_cmd_vel(linear_x, angular_z)
    self.last_angular_z = angular_z
    self.last_linear_x = linear_x

def _compute_distance_keeping_bbox(self, dt, aligned):
    ratio = self.bbox_dist_ratio
    if ratio is None or not aligned:
        self.prev_dist_err = 0.0
        return 0.0
    if self.latest_confidence < self.distance_min_confidence:
        self.prev_dist_err = 0.0
        return 0.0

    error = ratio - 1.0 # >0: farther than the reference, need to move
forward
    if abs(error) < self.bbox_ratio_tolerance:
        self.prev_dist_err = 0.0
        return 0.0
    deriv = (error - self.prev_dist_err) / dt
    linear_x = self.bbox_dist_kp * error + self.bbox_dist_kd * deriv
    linear_x = self.clamp(linear_x, -self.max_linear_x, self.max_linear_x)
    self.prev_dist_err = error

```

```

        return self.linear_dir * linear_x

def _compute_body_follow_pixel(self, error_x, half_w, dt):

    abs_err = abs(error_x)
    if abs_err >= self.no_ptz_engage_px:
        self.body_follow_armed = True
    elif abs_err <= self.pixel_tolerance:
        self.body_follow_armed = False

    if not self.body_follow_armed:
        self.prev_servo_err = 0.0
        return 0.0

    norm_x = self.clamp(error_x / half_w, -1.0, 1.0)
    deriv = (norm_x - self.prev_servo_err) / dt
    angular_z = self.body_dir * (self.no_ptz_yaw_kp * norm_x + self.no_ptz_yaw_kd *
deriv)
    angular_z = self.clamp(angular_z, -self.max_angular_z, self.max_angular_z)
    self.prev_servo_err = norm_x
    return angular_z

def _compute_body_follow(self, dt):

    if abs(self.current_x) >= self.servo1_engage_deg:
        self.body_follow_armed = True
    elif abs(self.current_x) <= self.body_tolerance_deg:
        self.body_follow_armed = False

    if not self.body_follow_armed:
        self.prev_servo_err = 0.0
        return 0.0

    servo_err = self.current_x
    deriv = (servo_err - self.prev_servo_err) / dt
    angular_z = self.body_dir * (self.body_kp * servo_err + self.body_kd * deriv)
    angular_z = self.clamp(angular_z, -self.max_angular_z, self.max_angular_z)
    self.prev_servo_err = servo_err
    return angular_z

```

6. Parameter Debugging

- Parameter file path:

```
$HOME/microros_ws/src/microros_v2_bringup/config/common.yaml
```

The parameters under the `visual_track` namespace in the parameter file `common.yaml` are divided into six groups by function. The role of each parameter and tuning suggestions are explained group by group below.

Group 1: Target centering / frame validity

Parameter	Default	Function	Tuning suggestion
<code>pixel_tolerance</code>	15	Pixel tolerance for target centering; error below this value is considered aligned	Smaller value makes the pan-tilt more accurate but may jitter; recommended 10-20
<code>max_step_xy</code>	5.0°	Maximum pan-tilt step angle per cycle	Larger value responds faster but may overshoot; recommended 3-8
<code>min_confidence</code>	0.01	Minimum target confidence	Below this value is considered lost; generally no adjustment needed
<code>bbox_max_age</code>	0.4s	Maximum valid latency of the bbox	Timeout is considered lost; can be appropriately increased when network latency is high

Group 2: Pan-tilt PID

Parameter	Default	Function	Tuning suggestion
<code>kp_x</code> / <code>kp_y</code>	0.6	Proportional gain, acting on the normalized pixel error	Increasing it responds faster; too large causes oscillation
<code>kd_x</code> / <code>kd_y</code>	0.15	Derivative gain, suppressing the error change rate	Increasing it reduces overshoot; too large is sensitive to noise

Group 3: ptz mode chassis follow

Parameter	Default	Function	Tuning suggestion
<code>enable_body_follow</code>	true	Whether to enable chassis follow steering	When disabled, the chassis does not turn and only the pan-tilt tracks
<code>body_kp</code>	0.03	Chassis follow proportional gain	Increasing it turns faster; too large causes swaying
<code>body_kd</code>	0.0	Chassis follow derivative gain	Generally keep 0; fine-tune when oscillation suppression is needed
<code>body_tolerance_deg</code>	3.0°	The chassis stops turning when the pan-tilt angle is below this value	Smaller value makes the chassis start more frequently
<code>servo1_engage_deg</code>	5.0°	The chassis starts turning when the pan-tilt angle exceeds this value	Must be greater than <code>body_tolerance_deg</code> to form hysteresis

Parameter	Default	Function	Tuning suggestion
<code>max_angular_z</code>	0.6 rad/s	Maximum chassis angular velocity	Adjust according to the site size; 0.3–0.4 recommended in narrow spaces
<code>reverse_body_direction</code>	false	Reverse the chassis steering direction	Enable when the direction is opposite on different models
<code>body_ff_gain</code>	1.0	Feedforward compensation gain of the servo for body rotation	1.0 full compensation, 0 disabled; generally keep 1.0

Group 4: no_ptz mode (no pan-tilt)

Parameter	Default	Function	Tuning suggestion
<code>no_ptz_yaw_kp</code>	0.9	Chassis rotation proportional gain	Increasing it turns faster; too large overshoots
<code>no_ptz_yaw_kd</code>	0.1	Chassis rotation derivative gain	Suppresses oscillation; generally 0.05–0.2
<code>no_ptz_engage_px</code>	30	Pixel error threshold for the chassis to start turning	Smaller value makes the chassis more sensitive
<code>no_ptz_align_px</code>	80	Pixel error threshold considered aligned	Forward/backward movement is only allowed below this value

Group 5: Distance keeping

Parameter	Default	Function	Tuning suggestion
<code>enable_distance_keeping</code>	true	Whether to enable distance keeping	When disabled, only turns without forward/backward movement
<code>distance_min_confidence</code>	0.6	Minimum confidence for forward/backward movement	Below this value only turns without moving, to prevent collision on misjudgment
<code>max_linear_x</code>	0.18 m/s	Maximum linear velocity	0.1 m/s recommended in narrow spaces
<code>distance_align_deg</code>	40°	In ptz mode, moves forward/backward only when the pan-tilt angle is below this value	Larger value triggers movement more easily but may be less safe

Parameter	Default	Function	Tuning suggestion
<code>reverse_linear</code>	false	Reverse the linear velocity direction	Enable when the +x direction is opposite on different models

Group 6: bbox area distance estimation

Parameter	Default	Function	Tuning suggestion
<code>bbox_ref_frames</code>	3	Number of reference sampling frames	Increasing it makes the reference more stable but initialization slower
<code>bbox_area_smooth</code>	0.4	EMA smoothing coefficient for the current area	Larger is more sensitive, smaller is smoother; recommended 0.3-0.6
<code>bbox_ratio_tolerance</code>	0.08	Relative distance dead zone	Larger value widens the dead zone, reducing fine-tuning but may be less accurate
<code>bbox_dist_kp</code>	0.5	Distance keeping proportional gain	Increasing it responds faster; too large causes oscillation
<code>bbox_dist_kd</code>	0.0	Distance keeping derivative gain	Fine-tune when suppressing distance-change oscillation
<code>bbox_ratio_max</code>	3.0	Relative distance ratio clamp	Prevents extreme distances from causing excessive velocity

```

visual_track:
  ros__parameters:
    # ---- Target centering / frame validity ----
    pixel_tolerance: 15          # Pixel tolerance for target centering (half side length
of the center grid)
    max_step_xy: 5.0             # Max pan-tilt step angle per cycle (degrees)
    min_confidence: 0.01        # Minimum target confidence; below this is considered
lost
    bbox_max_age: 0.4           # Max valid latency of the bbox (s); timeout is
considered lost

    # ---- Pan-tilt PID (acts on the normalized error [-1, 1]) ----
    kp_x: 0.6
    kd_x: 0.15
    kp_y: 0.6
    kd_y: 0.15

    # ---- ptz mode: chassis follow (front of the vehicle facing the target) ----
    enable_body_follow: true     # whether to enable chassis follow steering
    body_kp: 0.03               # Chassis follow proportional gain (acts on the servo1
angle)
    body_kd: 0.0                # Chassis follow derivative gain

```

```

body_tolerance_deg: 3.0      # Chassis stop-turning threshold (degrees)
servo1_engage_deg: 5.0      # Chassis start-turning threshold (degrees)
max_angular_z: 0.6          # Max angular velocity (rad/s)
reverse_body_direction: false # Reverse chassis steering (default: turn right when the
target is to the right)
body_ff_gain: 1.0            # Servo feedforward compensation gain for body rotation
(VOR): 1.0 full compensation, 0 disabled

# ---- no_ptz mode (no pan-tilt): chassis rotates to align by horizontal pixel error ---
-
no_ptz_yaw_kp: 0.9           # Chassis rotation proportional gain (acts on the
normalized pixel error)
no_ptz_yaw_kd: 0.1           # Chassis rotation derivative gain
no_ptz_engage_px: 30.0       # Pixel error threshold for the chassis to start turning
no_ptz_align_px: 80.0        # Pixel error threshold considered aligned
(forward/backward movement allowed)

# ---- Distance keeping (forward/backward movement, bbox circumscribed circle area
ranging, works for ptz / no_ptz) ----
enable_distance_keeping: true # whether to enable distance keeping
distance_min_confidence: 0.6  # Minimum confidence for forward/backward movement;
below this, only turns without moving forward/backward
max_linear_x: 0.18            # Max linear velocity (m/s)
distance_align_deg: 40.0      # ptz mode: move forward/backward only when |servo1| is
below this value (degrees)
reverse_linear: false         # Reverse linear velocity direction (the +x direction
may be opposite on different models)

# ---- bbox area distance estimation (near large, far small) ----
bbox_ref_frames: 3           # Reference sampling frame count (median of the first N
frames after capturing the target)
bbox_area_smooth: 0.4        # EMA smoothing coefficient for the current area (0~1,
larger is more sensitive)
bbox_ratio_tolerance: 0.08    # Relative distance dead zone (no movement when |ratio-
1| is below this value)
bbox_dist_kp: 0.5             # Distance keeping proportional gain
bbox_dist_kd: 0.0            # Distance keeping derivative gain
bbox_ratio_max: 3.0          # Relative distance ratio clamp

```

